

3.3 循环结构

3.3.1 for循环与while循环

- Python提供了两种基本的循环结构语句——for语句、while语句。
- for循环一般用于循环次数可以提前确定的情况，尤其是用于枚举序列等迭代对象或迭代器中的元素；
- while循环一般用于循环次数难以提前确定的情况，也可以用于循环次数确定的情况；
- 在循环次数确定的情况下，一般优先考虑使用for循环。
- 相同或不同的循环结构之间都可以互相嵌套，实现更为复杂的逻辑。

for 取值 in 序列或迭代对象：
 循环体

while 条件表达式：
 循环体

for循环

逐个取出数据
进行操作
直到结束

while循环

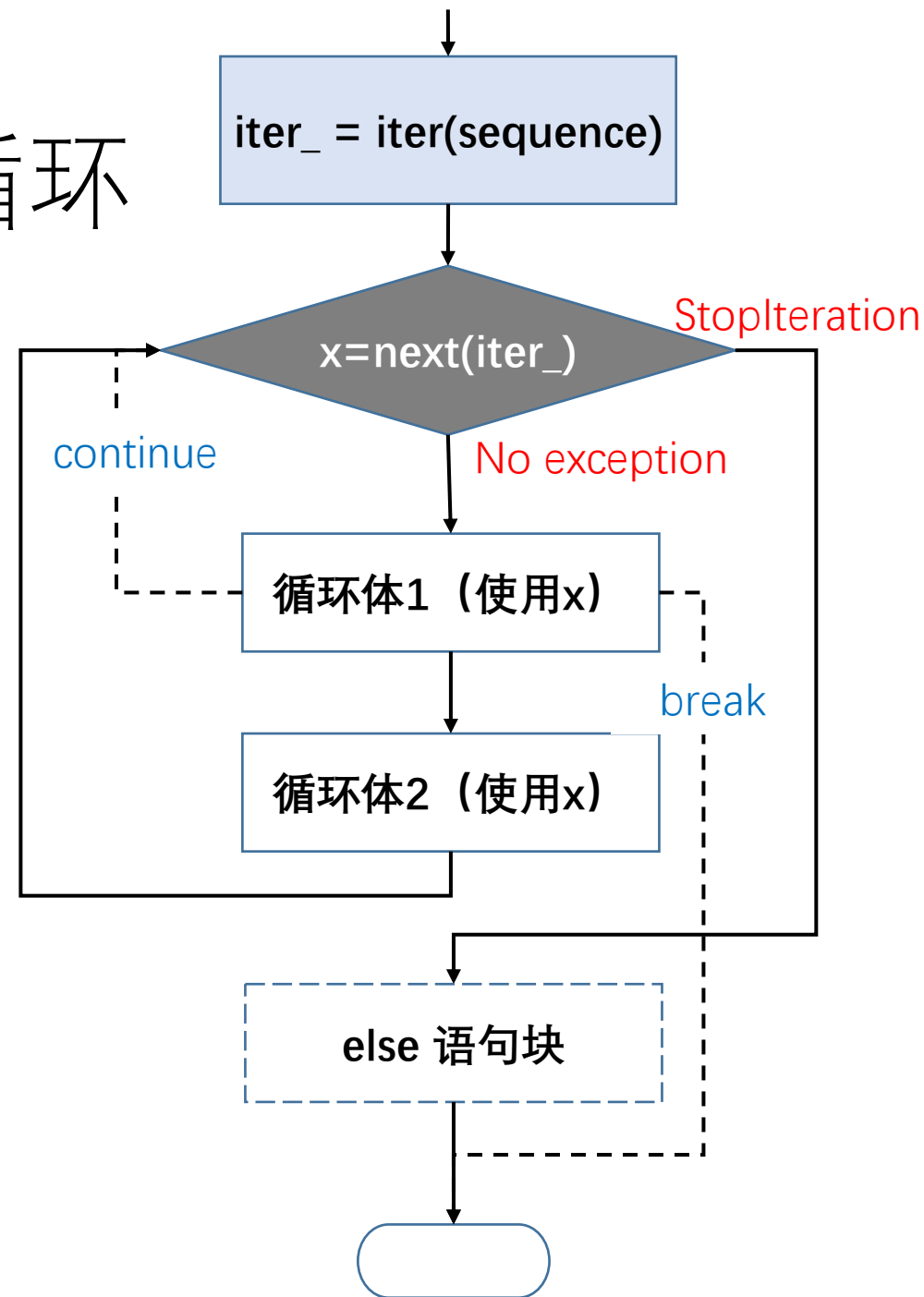
如果.....,

就.....

再次检查条件

3.3.1 for循环与while循环

```
for target in iterable: # Assign iterable
                        items to target
    循环体
    if 条件表达式1: break #可选
                        # Exit loop now, skip else if present
    if 条件表达式2: continue #可选
                        #go to top of loop now
else: #可选
    else子句 # If we didn't hit a 'break'
```



循环的设计 for

- 要解决一个问题，需要在一个计算空间内重复计算多次来求解
- 如果计算空间次数已知，可通过迭代对象描述。

for 循环变量 in 计算空间

进行计算

if 发现当前计算不会找到解： **continue 开始下一轮循环**

if 当前计算无需继续：

 如果已经找到解，且求解结束， **break 结束循环**

 **循环体其他后续语句**

else:

 继续计算 **如果不是break结束的循环**

程序其他后续语句

循环体



Review：可迭代对象、迭代器

- iterable:
 - 该对象包含一个`__iter__()`方法，通过调用其得到一个迭代器
 - 内置函数 `iter(iterable)`可以返回一个iterator
 - iterable对象包括： `str`,`list`,`tuple`,`dict`,`set`,`range`等
- iterator:
 - 迭代器一般也是可迭代对象，其`__iter__()`方法返回就是自身
 - 包含`__next__()`方法，调用其获得下一个元素
 - 内置函数`next(iterator)`从迭代器返回下一个元素，没有更多的元素时抛出异常 `StopIteration`
 - iterator包括：`reversed()`,`zip()`,`enumerate()` 等返回的对象
- **for var in iterator(or iterable) 循环**
 - 相当于每次取可迭代对象或者迭代器的下一个元素，执行一系列语句，然后取下一个元素执行，直到最后`StopIteration`时结束

3.3.1 for循环与while循环

for循环示例

- 例子：计算 $1+2+3+\dots+99+100$

```
def loop_for_1():  
    # total = 1 + 2 + 3 + 4 ... + 100  
    total = 0  
    for i in range(1, 101): #执行100次  
        total += i  
    print(total)
```

```
>>> loop_for_1()  
5050  
>>>
```

sum(range(1,101))

for循环示例（续）

3.5 例2：输出序列中的元素。

```
def for_loop_1():  
    a_list = ['a', 'b', 'mpilgrim', 'z',  
              'example']  
    for i, v in enumerate(a_list):  
        print('列表的第', i + 1, '个元素是:',  
              v)
```

列表的第 1 个元素是: a
列表的第 2 个元素是: b
列表的第 3 个元素是: mpilgrim
列表的第 4 个元素是: z
列表的第 5 个元素是: example

3.5 例3：求1~100之间能被7整除，但不能同时被5整除的所有整数。

```
def for_loop_2():  
    for i in range(1, 101):  
        if i % 7 == 0 and i % 5 != 0:  
            print(i, end=" ")
```

7 14 21 28 42 49 56 63 77 84 91 98



```
result = [ i for i in range(1,101) if i % 7 == 0 and i % 5 != 0]  
print('1~100之间能被7整除，但不能同时被5整除的所有整数:', *result)
```

for循环示例（续）：蒙特卡罗模拟计算pi

- 蒙特卡罗方法：如图所示，圆的面积为pi，而正方形的面积为4，在正方形的区域随机产生一个点，该点在圆内的概率为pi/4。

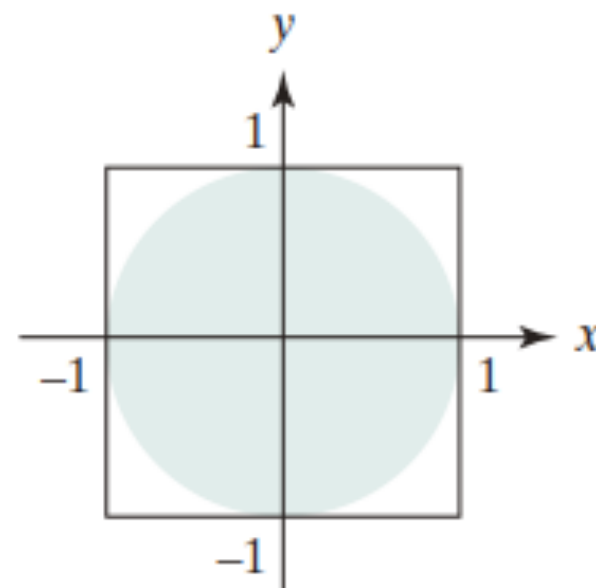
frequency = hits/tries = pi/4 → pi = 4*hits/tries

```
import random
def montecarlo_pi(tries = 1000000):
    hits = 0
    for i in range(tries):
        x = random.random() * 2 - 1
        y = random.random() * 2 - 1

        if x * x + y * y <= 1:
            hits += 1

    pi = 4 * hits / tries

    return pi
```



3.141392

循环嵌套(多重循环)

```
print('%d * %d = %d' % (i, j, i*j), end='\t')
```

3.5 例6：打印九九乘法表。

```
def print_multiplicatoin_table():  
    ''' 打印九九乘法表 '''  
    for i in range(1, 10):  
        for j in range(1, i + 1):  
            print(i, '*', j, '=', i * j, '\t', end=' ')  
        print()
```

外
循
环
i

1*1=1									
2*1=2	2*2=4								
3*1=3	3*2=6	3*3=9							
4*1=4	4*2=8	4*3=12	4*4=16						
5*1=5	5*2=10	5*3=15	5*4=20	5*5=25					
6*1=6	6*2=12	6*3=18	6*4=24	6*5=30	6*6=36				
7*1=7	7*2=14	7*3=21	7*4=28	7*5=35	7*6=42	7*7=49			
8*1=8	8*2=16	8*3=24	8*4=32	8*5=40	8*6=48	8*7=56	8*8=64		
9*1=9	9*2=18	9*3=27	9*4=36	9*5=45	9*6=54	9*7=63	9*8=72	9*9=81	

内循环j<=i

循环嵌套(续)

3.5 例10：编写程序，输出由1、2、3、4这四个数字组成的每位数都不相同的所有三位数。

```
def three_digit_numbers():  
    digits = (1, 2, 3, 4)  
    for i in digits:  
        for j in digits:  
            for k in digits:  
                if i != j and j != k and i != k:  
                    print(i * 100 + j * 10 + k)
```

123
124
132
134
142
143
213
214
231
234
241
243
312
314
321
324
341
342
412
413
421
423
431
432
...

循环嵌套(续)

所谓水仙花数是指1个3位的十进制数，其各位数字的立方和等于该数本身。153是水仙花数，因为 $153 = 1^3 + 5^3 + 3^3$

```
def find_narcissi_few():  
    '''输出“水仙花数”'''  
    for i in range(100, 1000):  
        ones = i % 10  
        tens = i // 10 % 10  
        hundreds = i // 100  
        if ones ** 3 + tens ** 3 + hundreds ** 3 == i:  
            print(i)
```

153
370
371
407

```
def find_narcissi_few2():  
    for hundreds in range(1, 10):  
        for tens in range(10):  
            for ones in range(10):  
                n = ones + 10*tens + 100*hundreds  
                if ones ** 3 + tens ** 3 + hundreds ** 3 == n:  
                    print(n)
```

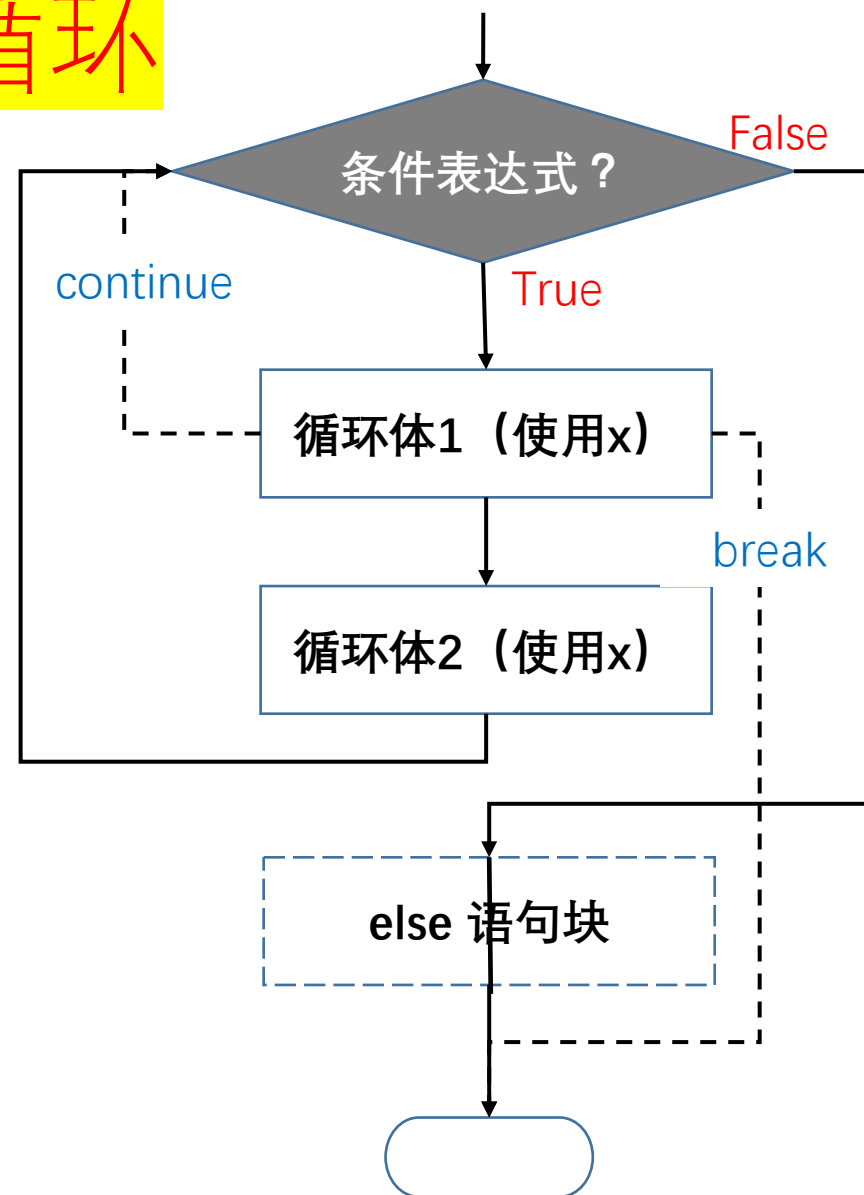
```
def find_narcissi_few3():  
    result = [i for i in range(100, 1000)  
              if (i % 10) ** 3 + (i // 10 % 10) ** 3 + (i // 100) ** 3 == i]  
    print('三位水仙花数:', *result)
```

列表解析式实现

3.3.1 for循环与while循环

- while循环和for循环都可以带else子句，当循环自然结束时（不是因为执行了break而结束）执行else结构中的语句。

```
while 条件表达式:  
    循环体  
    if 条件表达式1: break #可选  
    # Exit loop now, skip else if  
    present  
    if 条件表达式2: continue # 可选  
    # Go to top of loop now  
else: #可选  
    # Run if we didn't hit a 'break'  
    else子句
```



循环的设计while

- 要解决一个问题，需要在一个计算空间内重复计算多次来求解
- 循环变量决定了计算空间，初值设置以及判断是否是计算空间的最后一个取值

循环变量=初始值

while: 根据循环变量判断是否还在计算空间内，且求解未结束
进行计算

if 发现当前计算不会找到解： **continue 开始下一轮循环**

if 当前计算无需继续：

如果已经找到解，且求解结束，可以**break 结束循环**

……循环体其他后续语句

else:

继续计算**如果不是break结束的循环**

在下次循环前（包括continue之前）应该更新循环变量

为什么要引入while 循环？

```
def extract_digits_noloop(num):  
    """ 采用//和%运算来得到从右边开始数起的各个数字 """  
    place = 1 # 右边数起第几位  
    digit = num % 10  
    print('右边数起第%d位数字为%d' % (place, digit))  
    place = place + 1  
    num = num // 10  
    if num == 0:  
        return  
    digit = num % 10  
    print('右边数起第%d位数字为%d' % (place, digit))  
    place = place + 1  
    num = num // 10  
    if num == 0:  
        return  
    digit = num % 10  
    print('右边数起第%d位数字为%d' % (place, digit))  
    place = place + 1  
    num = num // 10  
    if num == 0:  
        return
```

```
def extract_digits(num):  
    """ 采用 //和 % 运算来得到从右边开始数起的各个数字 """  
  
    place = 1 # 右边数起第几位  
  
    while True:  
        digit = num % 10  
        print('右边数起第%d位数字为%d' % (place, digit))  
        place = place + 1  
        num = num // 10  
        if num == 0:  
            return
```

为什么要引入while 循环结构（续）？

```
def extract_digits(num):  
    """ 采用 //和 % 运算来得到从右边开始数起的各个数字"""  
  
    place = 1 # 右边数起第几位  
  
    while True:  
        digit = num % 10  
        print('右边数起第%d位数字为%d' %  
(place,digit))  
        place = place + 1  
        num = num // 10  
        if num == 0:  
            break
```



```
def sum_digits(num):  
    """ 采用 //和 % 运算来得到从右边开始数起的各个数字，然后求和"""  
  
    #place = 1 # 右边数起第几位  
    digits=[]  
    while num: # num != 0  
        digit = num % 10  
        digits.append(digit)  
        #place = place + 1  
        num = num // 10  
    return sum(digits)
```

3.3.1 for循环与while循环

while循环示例

- 计算 $1+2+3+\dots+99+100$

```
def loop_for_1():  
    # total = 1 + 2 + 3 + 4 ... + 100  
    total = 0  
    for i in range(1, 101): #执行100次  
        total += i  
    print(total)
```

sum(range(1,101))



```
def loop_while_1():  
    # total = 0+1 + 2 + 3 + 4 ... + 100  
    total = i = 0  
    while i <= 100: #执行101次  
        total += i  
        i += 1 #更新i, 影响条件表达式  
    # 循环结束, 打印结果  
    print(total)  
    '''  
    else: # 循环结束, 打印结果  
        print(total)  
    '''
```

while循环示例（续）

计算 $0! + 1! + 2! + 3! + \dots + 20!$

```
def sum_factorial(n):  
    '''计算0! + 1!+2!+3!+...+n! '''  
    i = item = 1  
    result = 1  
    while i <= n:  
        item *= i  
        result += item  
        i += 1  
  
    return result
```

```
>>> def sum_factorial(n):  
    '''计算0! + 1!+2!+3!+...+n! '''  
    i = item = 1  
    result = 1  
    while i <= n:  
        item *= i  
        result += item  
        i += 1  
    return result  
  
>>> sum_factorial(20)  
2561327494111820314  
>>> |
```

while循环示例（续）

- 例2：用户输入若干个分数，求所有分数的和。每输入一个分数后询问是否继续输入下一个分数。

```
def demo_3_2():
    endFlag = 'yes'
    s = 0
    while endFlag.lower() == 'yes':
        x = input("请输入一个正整数: ")
        x = eval(x)
        if isinstance(x, int) and
0<=x<=100:
            s = s + x
        else:
            print('不是数字或不符合要求')
            endFlag = input('继续输入? (yes
or no) ')
    print('整数之和=', s)
```

- 我们也可以在用户不输入任何信息（空字符串时）结束循环

```
def sum_demo2_3_2():
    s = 0
    x = input("请输入一个正整数: ")
    while x:
        x = eval(x)
        if isinstance(x, int) and 0<=x<=100:
            s += x
        else:
            print('不是数字或不符合要求')
            x = input("请输入一个正整数: ")

    print('整数之和=', s)
```

while循环示例 (续)

- 查找一个最小正整数，满足被3除余2，被5除余3，被7除余4

```
def min_number_while( ):
    ''' 查找一个最小正整数，满足被3除余2，被5除
    余3，被7除余4'''
    num = 0
    found = False    # 是否找到?
    while not found:
        num = num + 1
        if (num % 3 == 2 and num % 5 == 3
            and num % 7 == 4 ):
            found = True

    print('被3除余2，被5除余3，被7除余4的最小
    正整数是%d' % num)
```

```
def min_number2_while( ):
    ''' 查找一个最小正整数，满足被3除余2，被
    5除余3，被7除余4'''
    num = 1
    while not (num % 3 == 2 and num %
    5 == 3 and num % 7 == 4):
        num = num + 1

    print('被3除余2，被5除余3，被7除余4的最
    小正整数是%d' % num)
```

while循环示例 (续)

: 求多个解

- 查找前面n个正整数，满足被3除余2，被5除余3，被7除余4

```
def min_number_while( ):
    ''' 查找一个最小正整数，满足被3除余2，被5除
    余3，被7除余4'''
    num = 0
    found = False    # 是否找到?
    while not found:
        num = num + 1
        if (num % 3 == 2 and num % 5 == 3
            and num % 7 == 4 ):
            found = True

    print('被3除余2，被5除余3，被7除余4的最小
    正整数是%d' % num)
```



```
def first_n_number_while(total):
    num = 1
    count = 0    # 找到的个数
    first_n = []
    while count < total:
        if num % 3 == 2 and num % 5 == 3
        and num % 7 == 4:
            first_n.append(num)
            count += 1
            num = num + 1

    first_n = ' '.join([str(i) for i in
    first_n])
    print('被3除余2，被5除余3，被7除余4的前面%d
    个正整数是%s' % (total,first_n))
```

3.4 break和continue语句

- break语句在while循环和for循环中都可以使用，一般放在if选择结构中，一旦break语句被执行，将使得当前整个循环提前结束。
- continue语句的作用是终止当前循环，并忽略continue之后的语句，然后回到循环的顶端，提前进入下一次循环。
- 除非break或continue语句让代码更简单或更清晰，否则不要轻易使用。
 - 不要有太多的break或者continue!!!

break以及else示例

3.5 例7：求某整数（默认200）以内能被17整除的最大正整数。

```
def for_break_1(num=200):  
    ''' 求num(200) 以内能被17整除的最大正整数 '''  
    found = False  
    for i in range(num, 0, -1):  
        if i % 17 == 0:  
            # print(i)  
            print('%d以内能被17整除的最大正整数为%d' % (num, i))  
            found = True  
            break  
    if not found: #不以break退出  
        print('%d以内能被17整除的最大正整数无法找到' % num )
```



```
def for_break_else_1(num=200):  
    ''' 求num(200) 以内能被17整除的最大正整数 '''  
    for i in range(num, 0, -1):  
        if i % 17 == 0:  
            print('%d以内能被17整除的最大正整数为%d' % (num, i))  
            break  
    else: # not found:  
        print('%d以内能被17整除的最大正整数无法找到' % num )
```

break以及else示例

3.5 例8：判断一个数是否为素数。

```
def is_prime(n):  
    ''' 判断一个数是否为素数  
>>> n = int(input("Input a integer:"))  
>>> print(is_prime(n))  
    '''  
  
    import math  
    m = int(math.sqrt(n) + 1)  
    for i in range(2, m):  
        if n % i == 0:  
            ret = False  
            break  
    else: #为素数  
        ret = True  
    return ret
```

```
result = 'no' if 0 in [n % i for i in range(2, int(math.sqrt(n)+1))] else 'yes'
```

3.5 例9：鸡兔同笼问题。假设共有鸡、兔30只，脚90只，求鸡、兔各有多少只。

循环嵌套

```
def chicken_rabbit_cage(headers=30, feet=90):
    # 鸡兔同笼问题。假设共有鸡、兔30只，脚90只
    found = False
    for chickens in range(0, headers+1):
        for rabbits in range(headers+1):
            if ((chickens + rabbits) == headers
                and (2 * chickens + 4 * rabbits) == feet):
                print('鸡%d只,兔%d只' % (chickens, rabbits))
                found = True
                break
    if not found:
        print('无解')
```

```
def chicken_rabbit_cage2(headers=30, feet=90):
    # 鸡兔同笼问题。假设共有鸡、兔30只，脚90只
    for chickens in range(0, headers+1):
        if 2 * chickens + 4 * (headers - chickens) == feet:
            print('鸡%d只,兔%d只' % (chickens, headers - chickens))
            break
    else:
        print('无解')
```

```
result = [i for i in range(headers + 1)
           if 2 * i + (headers - i) * 4 == feet]
```

列表解析式实现

3.4 break和continue语句

1. 下面的代码用来计算小于100的最大素数，
请注意break语句和else子句的用法。

```
for n in range(100, 1, -1):  
    for i in range(2, n):  
        if n % i == 0: #n不是素数  
            break  
    else: #为素数  
        print(n)  
        break
```

退出内层循环

退出外层循环

输出：
97

2. 输出100以内的所有素数

• 相比左边代码，注意最后一个break语句去掉了。

```
for n in range(100, 1, -1):  
    for i in range(2, n):  
        if n%i == 0:  
            break  
    else: #如果缩进和if  
        print(n, end=' ')
```

循环中可选的else子句在循环结构不以break退出时调用。实践中要特别注意：

• 如果不小心for的else子句与for循环内的最后if语句缩进一致，语法正确但语义不同

输出：

97 89 83 79 73 71 67 61 59 53 47 43 41 37 31 29 23 19 17 13 11 7 5 3 2

while循环示例→ 引入break

- 查找一个最小正整数，满足被3除余2，被5除余3，被7除余4

```
def min_number_while( ):
    ''' 查找一个最小正整数，满足被3除余2，被5除余3，被7除余4'''
    num = 0
    found = False    # 是否找到?
    while not found:
        num = num + 1
        if (num % 3 == 2 and num % 5 == 3
            and num % 7 == 4 ):
            found = True

    print('被3除余2，被5除余3，被7除余4的最小正整数是%d' % num)
```



```
def min_number3_while( ):
    ''' 查找一个最小正整数，满足被3除余2，被5除余3，被7除余4'''
    num = 1
    while True:
        if num % 3 == 2 and num % 5 == 3 and num % 7 == 4:
            break;
        num = num + 1

    print('被3除余2，被5除余3，被7除余4的最小正整数是%d' % num)
```

3.4 break和continue语句：警惕continue可能带来的问题

```
#输出10以内的奇数
i=0
while i < 10:
    if i % 2==0:
        continue
    print(i, end=' ')
    i += 1
```

问题：continue时，i没有更新，死循环

```
#输出10以内的奇数
i= 0
while i< 10:
    i += 1
    if i % 2==0:
        continue
    print(i, end=' ')
```

严格意义上代码仍有问题！求的是[1,10]范围内的奇数

如果预先知道循环次数，**优先采用for循环**，更简洁，不易出错

```
#输出10以内的奇数
for i in range(10):
    if i % 2 == 0:
        continue
    print(i, end=' ')
```

```
#输出10以内的奇数
i= -1
while i < 9:
    i += 1
    if i % 2==0:
        continue
    print(i, end=' ')
```

i更新不起作用，for循环每次重新赋值

```
#输出10以内的奇数
for i in range(10):
    if i % 2 == 0:
        i += 1
        continue
    print(i, end=' ')
```



3.3.2 循环结构的优化

- 为了优化程序以获得更高的效率和运行速度
- 编写循环语句时，应尽量减少循环内部不必要的计算，将与循环变量无关的代码尽可能地提取到循环之外。
- 使用多重循环嵌套时，应尽量减少内层循环中不必要的计算，尽可能地向外提。

循环嵌套(续)

3.5 例10：编写程序，输出由1、2、3、4这四个数字组成的每位数都不相同的所有三位数。

```
def three_digit_numbers():  
    digits = (1, 2, 3, 4)  
    for i in digits:  
        for j in digits:  
            for k in digits:  
                if i != j and j != k and i != k:  
                    print(i * 100 + j * 10 + k)
```

```
>>>  
123 124 132 134 142  
143 213 214 231 234  
241 243 312 314 321  
324 341 342 412 413  
421 423 431 432  
>>>
```

加入代码，
每行输出最
多5个数

进行优化，部分判断提到外层循环，减少循环次数，



```
def three_digit_numbers2():  
    digits = (1, 2, 3, 4)  
    for i in digits:  
        for j in digits:  
            if i == j: continue  
            for k in digits:  
                if j != k and i != k:  
                    print(i * 100 + j * 10 + k)
```



```
def three_digit_numbers3():  
    counts = 0  
    digits = (1, 2, 3, 4)  
    for i in digits:  
        for j in digits:  
            if i == j: continue  
            for k in digits:  
                if j != k and i != k:  
                    print(i * 100 + j * 10 + k, end=' ')  
                    counts += 1  
                    if counts % 5 == 0: print()
```

3.3.2 循环结构的优化

一个三位数，由从1、2、3、4选择的数字组成，且三位数的数字允许重复，请输出这些三位数

```
import time
start = time.time()

digits = (1, 2, 3, 4)
for i in range(1000): # 重复1000次，计时有效
    result = []
    for i in digits:
        for j in digits:
            for k in digits:
                result.append(i*100+j*10+k)
print(result)

print(time.time()-start)
```

$i*100$ 和 $j*10$ 可以提到外层循环

```
start = time.time()

for i in range(1000):
    result = []
    for i in digits:
        i = i*100
        for j in digits:
            j = j*10
            for k in digits:
                result.append(i+j+k)
print(result)

print(time.time()-start)
```

3.3.2 循环结构的优化

- 另外，在循环中应尽量引用局部变量，因为局部变量的查询和访问速度比全局变量略快，在使用模块中的方法时，可以通过将其转换为局部变量来提高运行速度。

```
start = time.time() #获取当前时间
for i in range(10000000):
    math.sin(i)
print('Time Used:', time.time()-start) #输出所用时间

loc_sin = math.sin
start = time.time()
for i in range(10000000):
    loc_sin(i)
print('Time Used:', time.time()-start)
```

教材中xrange->range

3.5 案例精选 (课堂练习)

- 例5：求平均分。

```
score = [70, 90, 78, 85, 97, 94, 65, 80]
s = 0
for i in score:
    s += i
print(s / len(score))
```

```
print(sum(score) / len(score))
```

3.5 案例精选 (课堂练习)

- 例11：编写程序，生成一个含有20个随机数的列表，要求所有元素不相同，并且每个元素的值介于1到100之间。

```
x = []
while True: # while len(x) < 20:
    if len(x) == 20:
        break
    n = random.randint(1, 100)
    if n not in x:
        x.append(n)
print(x)
print(len(x))
print(sorted(x))
```

```
import random
x = random.sample(range(1, 100), 20)
print(len(x), '↑', sorted(x))
```

二维矩阵

- 可以通过列表的列表来表示二维矩阵matrix，即matrix的元素为数组的行列表，行列表中的元素为该行的各列
 - 二维矩阵的值可以通过行下标和列下标来访问，matrix[i][j]表示第i+1行和第j+1列的数据

```
matrix = [  
    [1, 2, 3, 4, 5],  
    [6, 7, 0, 0, 0],  
    [0, 1, 0, 0, 0],  
    [1, 0, 0, 0, 8],  
    [0, 0, 9, 0, 3],  
]
```

	[0]	[1]	[2]	[3]	[4]
[0]	1	2	3	4	5
[1]	6	7	0	0	0
[2]	0	1	0	0	0
[3]	1	0	0	0	8
[4]	0	0	9	0	3

```
matrix[0] → [1, 2, 3, 4, 5]  
matrix[1] → [6, 7, 0, 0, 0]  
matrix[2] → [0, 1, 0, 0, 0]  
matrix[3] → [1, 0, 0, 0, 8]  
matrix[4] → [0, 0, 9, 0, 3]
```

```
matrix[0][0] → 1  
matrix[4][4] → 3
```

二维矩阵的生成

```
import random
def generate_random_matrix(rows=3, cols = 3):
    matrix = [] # Create an empty list
    for row in range(rows):
        matrix.append([]) # Add an empty new row
        for column in range(cols):
            matrix[row].append(random.randint(0, 99))

    return matrix
```

二维矩阵的访问

```
def print_matrix(matrix):
    for row in matrix:
        for value in row:
            print(value, end = " ")
        print() # Print a new line
```

```
def print_matrix_2(matrix): # 通过下标访问
    for row in range(len(matrix)):
        for column in
range(len(matrix[row])):
            print(matrix[row][column], end =
" ")
        print() # Print a new line
```